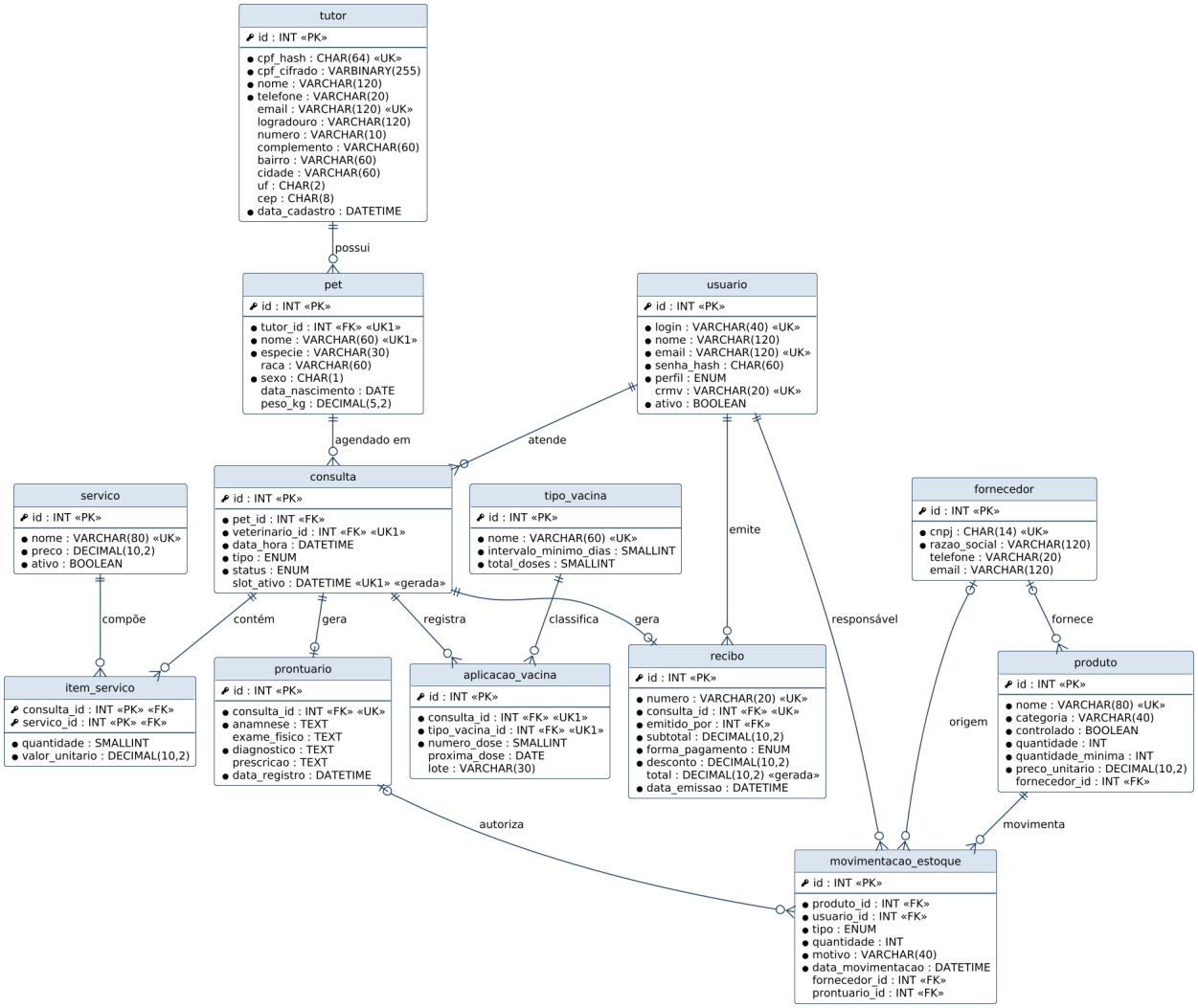
Fase B-3: Projeto Físico do Banco de Dados

8. Esquema Lógico do Banco de Dados Relacional (Fase B-3)

8.1. Diagrama do Esquema Lógico (notação pé-de-galinha)

O esquema lógico abaixo foi derivado do Diagrama de Classes (Seção 7) aplicando as regras de mapeamento objeto-relacional (Bezerra, Cap. 12) e as formas normais (1FN, 2FN e 3FN). O diagrama foi produzido em **PlantUML (notação IE / pé-de-galinha)** e o script DDL correspondente (Apêndice A) pode ser importado no **MySQL Workbench** (*File › Import › Reverse Engineer MySQL Create Script*) para regenerar o mesmo modelo na ferramenta.



Legenda: <<PK>> chave primária · <<FK>> chave estrangeira · <<UK>> chave única (identificador natural / chave candidata) · <<UK1>> chave única composta (todas as colunas marcadas com UK1 na mesma tabela formam uma única chave) · <<gerada>> coluna calculada pelo SGBD · ● coluna obrigatória (NOT NULL). Cardinalidades: || exatamente um, |o zero ou um, o{ zero ou muitos.

8.2. Relações (tabelas) e Restrições

Tabela	Chave primária	Identificador natural (UK)	Chaves estrangeiras	Principais restrições
tutor	id	cpf_hash; email	—	CPF guardado em duas colunas: hash (unicidade/busca) e cifrado (RN-08).
pet	id	(tutor_id, nome)	tutor_id → tutor	`tutor_id NOT NULL` (RN-01); `ON DELETE CASCADE` (composição); `sexo IN ('M','F')`.
usuario	id	login; email; crmv	—	`perfil` ∈ {RECEPCIONISTA, VETERINARIO, ADMINISTRADOR}; CHECK: CRMV obrigatório só para veterinário; `senha_hash` bcrypt (RNF-05).
consulta	id	(veterinario_id, slot_ativo)	pet_id → pet; veterinario_id → usuario	`status` ∈ {AGENDADA, REALIZADA, CANCELADA, NO_SHOW, PAGA}; impede dois atendimentos ativos do mesmo veterinário no mesmo horário; trigger RN-02.
servico	id	nome	—	`preco ≥ 0`.
item_servico	(consulta_id, servico_id)	—	consulta_id → consulta; servico_id → servico	`quantidade > 0`; `valor_unitario` guarda o preço praticado.
prontuario	id	consulta_id	consulta_id → consulta	0..1 prontuário por consulta (UK em consulta_id).
tipo_vacina	id	nome	—	`intervalo_minimo_dias > 0` (RN-04).

Tabela	Chave primária	Identificador natural (UK)	Chaves estrangeiras	Principais restrições
aplicacao_vacina	id	(consulta_id, tipo_vacina_id)	consulta_id → consulta; tipo_vacina_id → tipo_vacina	`numero_dose > 0`; `proxima_dose` alimenta o calendário (RF-08).
recibo	id	numero; consulta_id	consulta_id → consulta; emitido_por → usuario	`total = subtotal − desconto` (coluna gerada); CHECK do desconto PIX de 5% (RN-07).
fornecedor	id	cnpj	—	—
produto	id	nome	fornecedor_id → fornecedor	`quantidade ≥ 0`, `quantidade_minima ≥ 0`; visão `vw_produtos_estoque_baixo` (RN-06).
movimentacao_estoque	id	—	produto_id → produto; usuario_id → usuario; fornecedor_id → fornecedor; prontuario_id → prontuario	`quantidade > 0`; fornecedor só em ENTRADA; prescrição só em SAÍDA; trigger RN-05 e atualização do saldo.

8.3. Observações sobre o Esquema Relacional

a) Mapeamento das classes e associações

1. Cada classe persistente do Diagrama de Classes virou uma tabela com chave primária artificial (`id`, AUTO_INCREMENT). Os identificadores naturais apontados no modelo conceitual (CPF, login, CRMV, nome do produto, {nome + tutor} do pet) foram mantidos como **chaves únicas (UK)**, conforme o feedback da Fase A-2.
2. Associações **1:N** foram mapeadas com chave estrangeira no lado "muitos" (ex.: `pet.tutor_id`, `consulta.pet_id`).
3. Associações **1:0..1** (Consulta–Prontuario e Consulta–Recibo) foram mapeadas com FK + restrição `UNIQUE` no lado opcional, o que garante no máximo um prontuário e um recibo por consulta.
4. As **composições** do modelo de classes foram tratadas assim: Tutor–Pet usa `ON DELETE CASCADE`. Em Consulta–Prontuario/Recibo **não** foi usada exclusão em cascata, porque prontuário e recibo são documentos com valor legal/fiscal e não podem ser apagados junto com a consulta (o controle é feito pelo `status`).
5. O atributo composto `endereco` de Tutor foi decomposto em colunas atômicas (`logradouro`, `numero`, `bairro`, `cidade`, `uf`, `cep`) para atender à **1FN**.

b) Refinamentos em relação ao Diagrama de Classes (Fase A-2)

1. **Vacina → `tipo_vacina` + `aplicacao_vacina`.** Na classe `Vacina`, o nome da vacina se repetia a cada dose, e o intervalo mínimo entre doses (necessário para a RN-04) depende apenas do tipo de vacina, não da aplicação. Isso gera uma dependência transitiva (`aplicacao → nome → intervalo`), que viola a **3FN**. Por isso o catálogo foi separado em `tipo_vacina`, e cada dose aplicada fica em `aplicacao_vacina`.
2. **Remoção de redundâncias transitivas.** O diagrama de classes tinha associações diretas Pet–Prontuario, Pet–Vacina, Usuario–Vacina e Recibo–Tutor. No modelo relacional essas informações já podem ser obtidas pelo caminho via `consulta` (`prontuario → consulta → pet → tutor`). Manter essas FKs duplicadas violaria a 3FN e abriria espaço para inconsistências (ex.: prontuário apontando para um pet diferente do da consulta). Elas foram eliminadas. Pela mesma razão, `aplicacao_vacina` não guarda `data_aplicacao`: a data vem de `consulta.data_hora`.
3. **Serviços da consulta (`servico` e `item_servico`).** O UC-04 (passo 4) calcula o subtotal "com base nos serviços e procedimentos registrados", e o RF-10/UC-07 pede o relatório de "serviços mais realizados". O atributo único `Consulta.valor` não comporta isso. Por isso foi criada a relação N:N Consulta–Serviço, com a tabela associativa `item_servico`. A coluna `valor_unitario` **não** é redundância com `servico.preco`: ela guarda o preço vigente no dia do atendimento (dado histórico), que depende da chave inteira (consulta, serviço).
4. **Fornecedor.** No modelo de classes, `fornecedor` era uma *String* em Produto. Como o mesmo fornecedor se repete em vários produtos e entradas de estoque, ele virou a tabela `fornecedor`, com identificador natural `cnpj` (3FN).
5. **Usuário que emite o recibo** (`recibo.emitido_por`) foi adicionado para auditoria financeira.

c) Redundâncias controladas (desnormalizações intencionais)

1. `produto.quantidade` poderia ser calculada pela soma das movimentações, mas foi mantida para atender ao **RNF-02** (resposta em até 3 s) e permitir o alerta imediato de estoque mínimo (RN-06). A consistência é garantida pelos *triggers* `trg_mov_estoque_bi/ai`: o saldo só muda via `movimentacao_estoque`.
2. `recibo.subtotal` guarda uma "fotografia" do valor no momento da emissão (documento fiscal não pode mudar se o preço do serviço mudar depois). `recibo.total` é uma **coluna gerada** (`subtotal − desconto`), por isso nunca fica inconsistente.

d) Regras de negócio no esquema

Regra	Como é garantida no banco
RN-01	`pet.tutor_id NOT NULL` + FK.

Regra	Como é garantida no banco
RN-02	*Trigger* `trg_consulta_bi` (segunda a sábado, 8h–19h) + UK (`veterinario_id, slot_ativo`) contra conflito de horário. A coluna gerada `slot_ativo` vale `NULL` para consultas canceladas/no-show, liberando o horário.
RN-03	Domínio de `consulta.status` (inclui `NO_SHOW`). O cálculo das 2h é feito na camada de aplicação (UC-02).
RN-04	`tipo_vacina.intervalo_minimo_dias`, validado pela aplicação antes de inserir em `aplicacao_vacina` (UC-03, FA-01).
RN-05	`movimentacao_estoque.prontuario_id` + *trigger* que bloqueia saída de produto controlado sem prescrição.
RN-06	`produto.quantidade_minima` + visão `vw_produtos_estoque_baixo`.
RN-07	CHECK `ck_recibo_desconto_pix` (desconto = 5% do subtotal se PIX, senão 0).
RN-08 / RNF-05	`tutor.cpf_cifrado` (AES-256 na aplicação) + `tutor.cpf_hash` (SHA-256 para unicidade e busca, sem expor o CPF) e `usuario.senha_hash` (bcrypt).

e) Relação com o protótipo NoSQL

O protótipo em MongoDB, implantado na VPS Oracle Cloud nas fases anteriores, foi um estudo de modelagem orientada a documentos. Para o sistema em produção foi adotado o modelo **relacional**, porque o domínio exige integridade referencial forte (pet ↔ tutor, consulta ↔ recibo), transações envolvendo várias entidades (registrar uma saída de estoque e atualizar o saldo) e restrições declarativas (UK, CHECK) que garantem as regras de negócio.